Pack 06 · Model internals

The transformer details interviewers probe: attention math, masking, the KV cache, sampling and normalization. Lessons 05, 07, 10–14, 17. Answer each question out loud in about a minute, then check the card.

Q1 · Why ÷ √d, and why subtract the max in softmax? PRIORITY

Lessons: 05, 11

> **INTERVIEWER**
>
> Attention computes q · k ÷ √d before softmax. What goes wrong without ÷ √d (d = 128)? Softmax kernels subtract the row's maximum before e ^ x: why, and does it change the result?

60-second answer

- A dot product of two d-dimensional vectors with unit-variance entries has variance d, so scores spread by **√d ≈ 11** for d = 128.
- Without scaling, softmax becomes nearly **one-hot**: attention locks onto one token and gradients vanish. ÷ √d restores a spread of about 1. It rescales the spread; it doesn't center anything.
- e ^ x **overflows** quickly in FP16 (max 65,504, so e ^ 12 is already too big). Subtracting the max keeps every exponent ≤ 0.
- It **doesn't change the result**: softmax(x − c) = softmax(x), since e ^ (−c) cancels top and bottom.

Key numbers: $\text{softmax}(x)_i = e^{(x_i - \max)} \div \Sigma_j e^{(x_j - \max)}$.

Likely follow-up: "FlashAttention sees scores in blocks. How does it get an exact softmax?" Online softmax: keep a running max and sum; when the max grows, rescale the old sum and the output accumulator by e ^ (m_old − m_new).

> **TRAP**
>
> I said √d "centers the mean". In the online-softmax code I forgot the running max must never decrease, and forgot to rescale the accumulator.

Q2 · The causal mask, and why the KV cache is valid SOLID

Lessons: 11, 14, 17

> **INTERVIEWER**
>
> Future positions are masked by adding −∞ before softmax: why −∞ and not 0? Why do earlier rows of the score grid never need recomputing in decode, and how much work does the naive "re-feed everything" loop waste for a 512-token prompt and 256-token reply?

60-second answer

- e ^ (−∞) = 0, so masked tokens get exactly zero weight. Adding 0 would leave $e^0 = 1$, a full weight on the future.
- A new token sits in every earlier row's **masked future**, so earlier rows, and the K and V computed for earlier tokens, **never change**. That's why caching K and V is exact.
- Naive loop: N × P + N(N − 1) ÷ 2 = 256 × 512 + 32,640 ≈ **164k token passes**, versus P + N = **768** needed: ≈ 213× waste.
- The KV cache turns decode into "one new token attends to the cached past".

Key numbers: naive work = N × P + N(N − 1) ÷ 2; needed = P + N.

Likely follow-up: "What does the cache cost?" Memory: 128 KiB per token for Llama 3 8B, which is what limits concurrency (pack 02).

> **TRAP**
>
> Give the reason, not just the fact: $e^0 = 1$ for why 0 fails, and "the new token is in their masked future" for why old rows stay valid.

Q3 · Three KV cache bugs, and a broken decode loop PRIORITY

Lessons: 10, 13, 17

> **INTERVIEWER**
>
> A preallocated KV cache shows three bugs: (1) the first token is fine, the rest gibberish; (2) outputs are subtly worse, with the zero-filled buffer read in full; (3) an answer mentions the previous user's conversation. And a decode loop uses `out.logits[:, 0, :].argmax(-1)` and ignores the prompt. What's wrong in each?

60-second answer

- **Bug 1, wrong position:** decode tokens are encoded at the wrong position (RoPE not advanced). Prefill was right, so the first token is fine.
- **Bug 2, unfilled slots:** attention reads empty zero slots; zero keys still get weight after softmax. Attend only to the filled length.
- **Bug 3, no reset:** the cache isn't cleared between requests. A correctness bug and a **privacy** bug.
- **Decode loop:** logits are **(batch, T, vocab)**; position 0 predicts what follows the first token. Use `logits[:, -1, :]`. Engines compute only the last row: full FP32 logits for 8,192 tokens are 8,192 × 128,256 × 4 B ≈ **4.2 GB**, versus 0.5 MB for one row.

Key numbers: logits shape (batch, T, vocab); sample from the last position.

Likely follow-up: "How would you test a KV cache?" Compare the cached loop's outputs with the naive loop token by token; they must match exactly.

> **TRAP**
>
> I guessed "wrong K/V stored" for bug 1 (that would break the first decoded token too), and said logits are hidden × vocab (that's the LM head's weight matrix).

Q4 · Repeating stories vs bizarre words PRIORITY

Lessons: 13

> **INTERVIEWER**
>
> A story assistant at temperature 0 repeats the same phrase in long stories. Switching to temperature 1.2 with no other settings gives occasional bizarre words. Cause and fix for each?

60-second answer

- **Greedy** always takes the top token, so once a phrase becomes most likely it loops. Use sampling (temperature around 0.7–1.0), optionally a repetition penalty.
- **Temperature > 1 flattens** the distribution, lifting the long tail of junk tokens. Cut the tail with **top-p** (smallest set reaching probability p) or **min-p** (drop tokens below a fraction of the top one).
- Order: **temperature → softmax → cut the tail → renormalize → draw**.

Key numbers: p ∝ e^(logit ÷ T); T → 0 is greedy.

Likely follow-up: "Why is min-p often better than top-p at high temperature?" It scales with the top token's confidence, so it cuts hard when the model is sure and stays open when it isn't.

> **TRAP**
>
> I said "cut the tail, then apply temperature" (it's the other way round) and that temperature equalizes outcomes (it flattens).

Q5 · The residual stream and what RMSNorm normalizes PRIORITY

Lessons: 12

> **INTERVIEWER**
>
> Why does each layer do x ← x + f(x) instead of x ← f(x)? Does RMSNorm normalize the residual stream itself, and why does that matter?

60-second answer

- The residual connection gives an **identity path**: gradients flow straight through deep stacks, and each layer only adds a refinement.
- Modern models are **pre-norm**: x ← x + f(RMSNorm(x)). RMSNorm normalizes only the **copy fed into each sublayer**; the stream stays raw and accumulates.

- RMSNorm divides by the root mean square and applies a learned gain, with **no mean subtraction**, which makes it cheaper than LayerNorm.

Key numbers: $\text{RMSNorm}(x) = x \div \sqrt{(\text{mean}(x^2) + \varepsilon)} \times g$.

Likely follow-up: "Where does the final norm go?" One last RMSNorm before the LM head, since the raw stream has grown through all the layers.

> **TRAP**
>
> I didn't recall that only the sublayer input is normalized, not the stream.

Build it yourself

- `exercises/05_07_online_softmax.py`: exact softmax computed block by block, then the attention output in one pass, as FlashAttention does. Run `uv run exercises/05_07_online_softmax.py`.
- `exercises/10_13_sampler.py`: a sampler from raw logits (greedy, temperature, top-p, min-p, renormalize, draw). Run `uv run exercises/10_13_sampler.py`.
- `exercises/14_17_kv_cache.py`: a cached attention loop with preallocated buffers, proven to match the naive loop exactly, plus a demo of the unfilled-slot bug. Run `uv run exercises/14_17_kv_cache.py`.